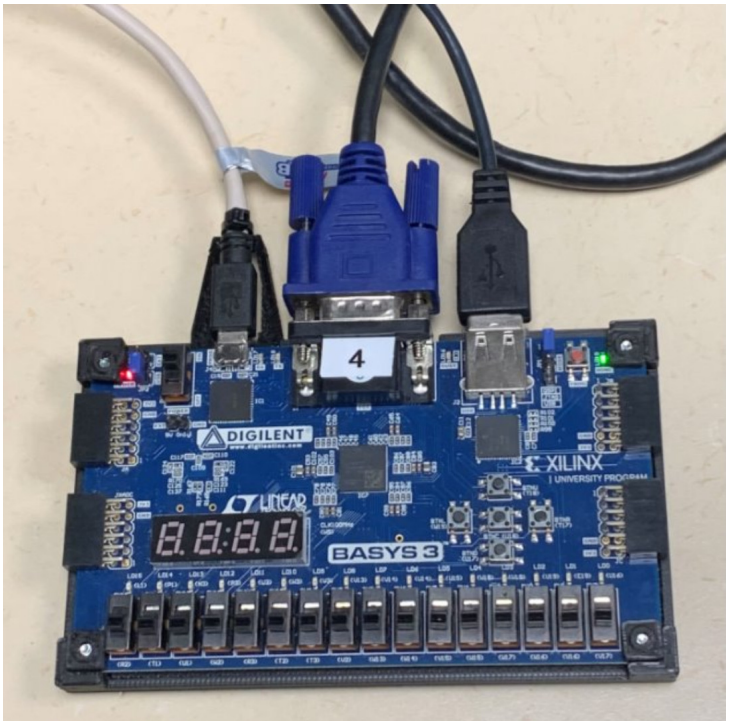
Teknisk dokumentation

TSEA83

Embla Olin
Ivar Lenner
Lovisa Bogeklint
Frida Strömberg

Examinator: Anders Nilsson
Handledare: Theodor Lindberg



Innehållsförteckning

Innehållsförteckning **1**
1. Inledning **3**
1.1 Bakgrund 3
1.2 Syfte 3
1.3 Källor 3
2. Apparaten **3**
3. Teori **4**
3.1 PS/2-modulen 4
3.2 VGA-modulen 4
3.2.1 VGA-standardens upplösning och timing 4
3.2.2 Horisontell och vertikal räknare 5
3.2.3 Synkronisering och blankning 5
3.2.4 Adressberäkning för framebuffer och tecken-ROM 5
3.2.5 Färgkodning och utmatning 6
3.3 ALU-modulen 6
3.3.1 Operandsval och datavägar 6
3.3.2 Aritmetiska operationer 6
3.3.3 Bitoperationer och skiftoperationer 6
3.3.4 Flagghantering 7
3.4 Othello-logik 7
3.4.1 Början av spelet och dess mål 7
3.4.2 Brickvändning och regler kring drag 8
3.4.3 Spelets slut 8
4. Hårdvaran **8**
4.1 Systemöversikt 8
4.2 PipeCPU 9
4.2.1 Programminne (PM) 9
4.2.2 Registerminne (RM) 9
4.2.5 Aritmetisk-logisk enhet (ALU) 9
4.2.6 Kontrollenheten (CU) 10
4.3 Dataminnet (DM) 10
4.4 PS/2-tangentbordet (KBD_comp) 10
4.5 VGA-motor 10
4.5.1 Allmänt om VGA-motorn 10
4.5.2 Signalkopplingar och dataströmmar 10
4.6 Sammanfattning 11
5. Mjukvaran **11**
5.1 Beskrivning 11
5.2 Översikt 11
5.3 Instruktionsuppsättning 11
5.4 Spelbrädesinitialisering 11

5.5 Interaktion och tangentbordsinmatning....12
5.6 Brickplacering och spelmekanik....12
5.7 Sammanfattning....13
6. Slutsatser....13
7. Referenser....14
7.1 Appendix....14
7.1.1 VHDL-kod....14
7.2.1 Instruktionsuppsättning....14
Data-förflyttning:....14
Aritmetik:....14
Bit-operationer:....15
Kontrollflöde:....16
7.2.2 Blockscheman....16

1. Inledning

1.1 Bakgrund

I detta projekt har en egenkonstruerad dator designats och implementerats på ett FPGA (Basys3) med avseende att köra brädspelet Othello helt fristående. Datorn består av en 16-bitars processor med tvåstegs-pipeline och har bland annat ett uppdelat program- och dataminne, en kontrollenhet (CU), samt en aritmetisk-logisk enhet (ALU). För indata används ett PS/2-tangentbord och för utdata används en VGA-skärm som uppdateras vid varje steg i spelet. Projektet ingår i kursen Datorkonstruktion (TSEA83) vid Linköpings tekniska universitet och bygger vidare på laborationer, där bland annat en VGA-motor och PS/2-gränssnitt implementerats. Genom att kombinera dessa moduler med en egenutvecklad CPU ges en helhet där processor, minne samt in- och utmatning samverkar för att realisera ett interaktivt spel.

1.2 Syfte

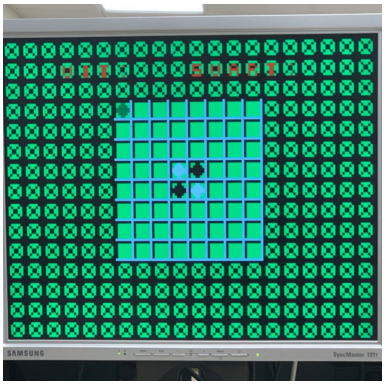
Syftet med projektet var att planera och utveckla en datorkonstruktion samt att samarbeta i en projektgrupp.

1.3 Källor

- Egen designspecifikation för Othello-processorn
- Kurslitteratur i VHDL och digitalteknik
- PS/2-dokumentation
- VGA-dokumentation och labbmaterial från kursmoment i grafik

2. Apparaten

Apparaten som konstruerats är en dator vars enda uppgift är att kunna köra ett egenkonstruerat Othello-spel. För att uppleva spelets utseende, se figur 1.



Figur 1

Spelet spelas på ett tangentbord kopplat till ett Basys3-kort. Tangenterna som används för spelet är WASD och enter. WASD väljer var på brädet som brickan ska placeras; W flyttar uppåt, A åt vänster, S nedåt och D åt höger. Enter placerar brickan, vilket resulterar i att brickor vänds och spelet övergår till andra spelarens tur.

3. Teori

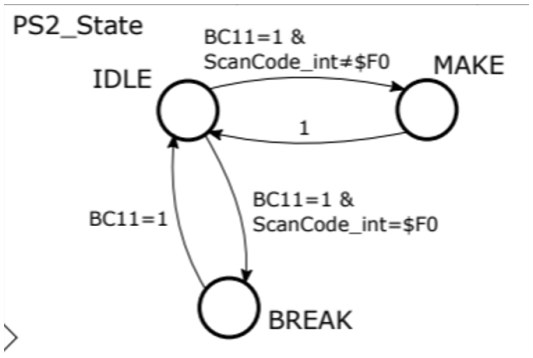
3.1 PS/2-modulen

PS/2-protokollet används för att upprätthålla en seriell kommunikation mellan tangentbordet och CPU:n. Den använder sig av en data- och en klocksignal, där data skiftas in på datalinjen 1 bit i taget vid klockans negativa flank, för att säkerställa korrekt mottagning av datan.

Varje tecken som skickas in i PS/2-modulen är 11 bitar stort. Dessa 11 bitar består av en startbit följt av 8 databitar samt en partitetsbit och en stoppbit.

PS/2-komponenten behöver hålla koll på vilka bitar som tillhör ett tecken. Detta görs med en 11-bit räknare som räknar upp till 11 bitar. Då 11 bitar lästs in, finns en färdig scancode och tangenttryckningen har registrerats. Detta tillstånd kallas för MAKE (se hela tillståndsdiagrammet i figur XX nedan). MAKE triggar CPU:ns I-flagga, som används för att signalera när en tangenttryckning ska lagras i registerminnet. Då I-flaggan är hög, skickas signalen till register 15 i CPU:ns registerminne, som enbart lagrar tangenttryckningar.

Tangenttryckningarna tolkas i programkoden där man vid en läsning av register 15, får ut till exempel “1D” (scancode för tangenten W), vilket innebär att spelmarkören ska flyttas uppåt. Tabell 1 beskriver scancodes för de tangenter som används i projektet.



Figur 2. Tagen från labbkompendium för laboration 4.

Tangent	Scancode
W	1D
A	1C
S	1B
D	23
Enter	5A

Tabell 1.

3.2 VGA-modulen

3.2.1 VGA-standardens upplösning och timing

Standard VGA använder 640x480 pixlar vid 60 Hz bilduppdatering. För att detta ska kunna uppdateras behövs en pixelklocka på 25 MHz och en rad- och bildscanning. Det ser ut som följande, i tabell 2:

Fas	**Pixlar (Horisontellt)**	**Rader (Vertikalt)**

Aktiv	640	480
Front porch	16	10
Sync pulse	96	2
Back porch	48	33
Totalt	800	525

Tabell 2.

Aktiv fas är den tid då pixeldata visas. Front porch innebär ett kort mellanrum före synkpulsen och synkpulsen är en låg nivå som signalerar början av nästa rad (Hsync) eller bild (Vsync). Back porch motsvarar tiden efter synkpulsen fram till början av nästa aktiva rad eller bild. Timingen kräver 800 pixlar per rad och 525 rader per bildruta, vilket ger VGA:s totala bildmönster.

3.2.2 Horisontell och vertikal räknare

För att ta fram Hsync och Vsync och blankning används två räknare, Xpixel och Ypixel. Xpixel räknar pixelklockor inom en rad från 0 till 799. Då den har nått 799 (totalt antal pixelklockor minus 1) återställs den till 0. Under Xpixel mellan 0 till 639 har vi den aktiva fasen (bildvisning). När Xpixel befinner sig mellan 656 och 751 blir Hsync låg. Därefter sker blankning där Xpixel sätts till hög ≥ 640 eller Ypixel ≥ 480.

Ypixel befinner sig mellan 0 och 525 och räknar raderna. Varje gång Xpixel återställs ökar Ypixel med 1. När Ypixel sedan når 524 återställs den till 0. När Ypixel befinner sig mellan 0 och 479 är den aktiva fasen vertikal. Ypixel kan också befinna sig mellan t.ex 490 och 491 och är då vertikal inom sync-pulsen och Vsync blir låg. Dessa räknare skapas i separata VHDL-processer som är synkroniserade mot pixelklockan.

3.2.3 Synkronisering och blankning

Hsync och Vsync är digitala pulser som styr skanningen i bildskärmen. Vanlig polaritet är aktiv låg. Bland är en funktion som används för att maskera utsignal (placera färg till svart bakgrund) under porches och sync-faser.

3.2.4 Adressberäkning för framebuffer och tecken-ROM

I designen finns en enkel teckenbaserad grafik. Där används en videominnesadress som är en 15-bitars vektor som kombinerar vilken teckenplats (80x60 matriser) på skärmen som ska adresseras (baserat på pixelräknaren i X-led samt pixelräknaren i Y-led). I designen används också en adress för teckenminnet som blir det värde som kommer från videominnet. Teckenminnet är ett enkelt enportat minne som endast går att läsa från och är definierad som ett minne med 2048 rader som i sin tur är 12 bitar breda. Varje rad är ett 12-bitars färgkod som sedan blir utdatan till VGA-skärmen, det vill säga 4 bitar per färg: Röd, Grön och Blå.

3.2.5 Färgkodning och utmatning

Teckenminnet innehåller som nämnt tidigare en 12-bitars färgkod. När signalen blank är låg kopplas dessa bitar ut och blir utsignalen till VGA-skärmen och då signalen blank är hög sätts utsignalen till koden för svart.

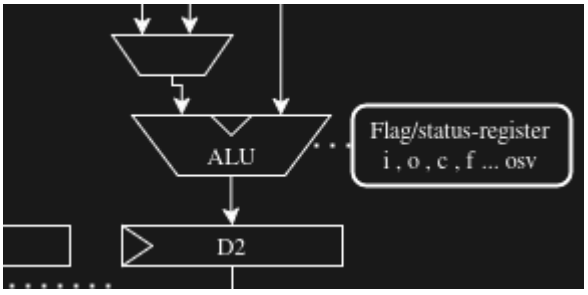
Med denna teori som grund, kan man förstå och vidareutveckla VGA-modulen (genom att till exempel lägga till sprites, texter, animationer eller dubbla buffertar).

3.3 ALU-modulen

ALU:n ansvarar för att utföra samtliga aritmetiska och logiska operationer i instruktionerna, och löser allt kring statusflaggorna samt styr vilka hopp som ska göras och jämför dessa.

3.3.1 Operandsval och datavägar

ALU:n har två 16-bitars operandingångar, men har en närliggande multiplexer som behöver välja mellan ett registervärde eller en konstant ifrån instruktionsordet. Denna multiplexer får sin kontrollsignal ifrån kontrollenheten som i sin tur är baserat på instruktionskoden i instruktionsordet. Detta tydliggörs genom figur 4.



Figur 4

3.3.2 Aritmetiska operationer

De aritmetiska operationerna bygger på en två komplements-representation, vilket gör att subtraktion genomförs som addition av den inverterade andra operanden adderat med värdet 1. ALU:n utför operationer baserat på kontrollsignalerna som i sin tur kommer från kontrollenheten som i sin tur är baserade på instruktionen i IR1.

I ALU:n beräknas t.ex ett 17-bitars temporärt resultat vid addition och subtraktion (för att veta om flaggan C ska uppdateras), medan multiplikationens resultat är 16 bitar.

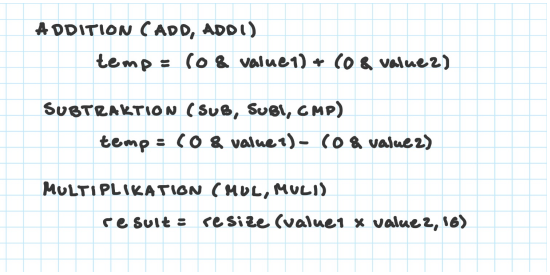
3.3.3 Bitoperationer och skiftoperationer

ALU:n har även andra funktioner som bland annat:

- Logiskt vänsterskift (LSL)
- Logiskt högerskift (LSR)
- Bitvis-AND
- Bitvis-OR

- Bitvis-NOT

Varje operation har ett unikt kodvärde som cLSL, cLSR, cAND, cOR, cNOT i kontrollsignalerna.



Figur 5.

3.3.4 Flagghantering

Efter att ett resultat har beräknats uppdateras fyra status-flaggor som ligger i ett separat flaggregister och styr bland annat villkorliga hopp.

- Carry-flaggan (C)

C-flaggan sätts till resultatet av vad biten på index 16 i det temporära resultatet för ADD och SUB var; Men endast då ALU:n får kontrollsignalerna att göra addition eller subtraktion, i annat fall ska inte C-flaggan uppdateras.

- Overflow-flaggan (V)

V-flaggan räknas ut med XOR-logik. Overflow-flaggan gäller bara om det är en ADD- eller en SUB-operation.

- Negativ-flaggan (N)

N-flaggan är endast satt om resultatets mest signifikanta bit är 1.

- Noll-flaggan (Z)

Z-flaggan är endast satt om hela resultatet är 0.

- Avbrotts-flaggan (I)

Avbrottsflaggan är endast satt om CPU-komponenten tagit emot en avbrottssignal från tangentbordskomponenten.

Dessa flaggor tillsammans utgör grunden för instruktionerna “BEQ”, “BNE”, “CMP” samt “BRIE” där kontrollenheten undersöker Z-status och V-status för att avgöra om ett hopp ska ske. ALU-modulen i vår processor ger en motor för beräkningar och jämförelser i Othello-programmet. Den kombinerar operand-multiplexing med uppsättningar av aritmetiska och logiska operationer, samt hanterar flaggorna, vilket utgör en grund för spelet.

3.4 Othello-logik

3.4.1 Början av spelet och dess mål

Vid starten av ett nytt spel har spelarna 32 dubbelsidiga brickor vardera. Spelarna lägger ut 2 brickor vardera på spelplanen i en kvadrat för att utgöra början av spelet. Målet med spelet är sedan att ha majoriteten av brickorna på spelbrädet i sin egen färg, alltså svart eller vit.

3.4.2 Brickvändning och regler kring drag

För att vända på brickor, vilket krävs för att vinna spelet, måste man utflanka sin motståndares brickor. Det vill säga om vit spelare ska spela och planen visar en vit bricka med tre svarta brickor i en rad bredvid sig så kan vit spelare lägga en vit bricka på andra sidan av dessa för att sedan vända alla. Man får vända brickor vertikalt, horisontellt och diagonalt.

Spelaren får endast lägga brickor som resulterar i vändningar, detta innebär att det övergår till den andra spelarens tur om det inte finns några drag som resulterar i brickvändningar.

3.4.3 Spelets slut

Spelet är slut när ingen av spelarna kan lägga brickor längre, då vinner den med flest av sin egen färg på brädet. Spelet kan alltså ta slut innan båda spelarna lagt sina 30 brickor.

4. Hårdvaran

I följande kapitel beskrivs datorns hårdvarukomponenter. Varje modul redovisas med dess anslutningar, interna arkitektur och hur de samverkar med de övriga delarna i systemet.

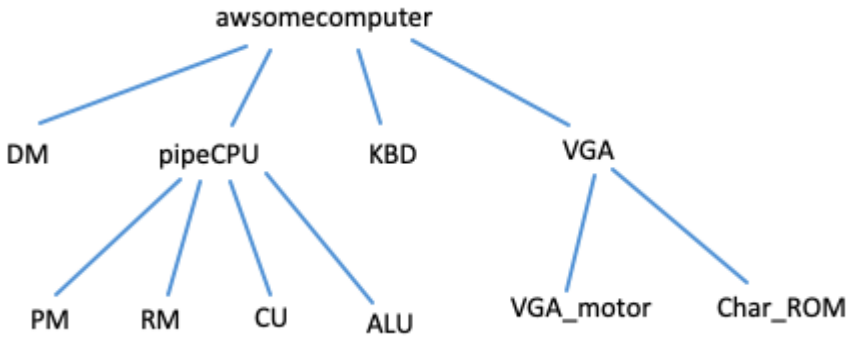
4.1 Systemöversikt

Systemet är baserat på en tvåstegs-pipelined CPU (pipeCPU) som är implementerad på ett Basys3 FPGA. Hårdvaran består av ett antal komponenter, inklusive:

- **Programminne (PM)**
  Innehåller de instruktioner som ska exekveras. Med andra ord maskinkoden för Othelloapplikationen.
- **PipeCPU**
  PipeCPU:n kopplar ihop CU, RM, ALU och PM.
- **Kontrollenhet (CU)**
  Kontrollenheten genererar styrsignaler baserat på information i instruktionerna.
- **Registerfil (RM)**
  Registerfilen har 16 register där alla register är 16 bitar stora.
- **ALU**
  ALU:n utför aritmetiska och logiska operationer och uppdaterar statusflaggor.
- **Dataminne (DM)**
  Dataminnet används som lagring för variabler och spelbrädet. Vilket är delat mellan CPU och VGA.
- **PS/2-tangentbord (KBD_comp)**
  PS/2-tangentbordet avkodar scancodes och genererar avbrottspulser.
- **VGA-motor (VGA_comp + char_rom)**
  VGA-motorn läser in video-ram och slår upp teckenbilder och genererar HS/VS/RGB

Alla moduler synkroniseras mot en gemensam 100 MHz-klocka (clk) med aktiv hög reset.

Systemet är uppbyggt enligt en fil hierarki som visas i nedanstående figur, där varje komponent ligger i sin egna fil/katalog. Projektets “toppfil” heter “awesomecomputer.vhd” och fungerar som den huvudsakliga konstruktionen som syntetiseras.



Figur 8.

4.2 PipeCPU

PipeCPU_comp är den övergripande enheten som integrerar processorns centrala delar. Komponenten innefattar programminnet, registerfilen, kontrollenheten och ALU:n. Den ansvarar för att hämta instruktioner från PM och hantera exekvering i två pipeline-steg: Fetch och execute. I fetch-steget laddas instruktioner in i register IR1 och tolkas av kontrollenheten. I execute-steget överförs instruktionen till registret IR2, där ALU:n exekverar operationen baserat på styrsignalerna. PipeCPU hanterar dessutom villkorliga och ovillkorliga hopp via signalen jump_condition, som bestäms av instruktionens innehåll och CPU:ns statusflaggor. För att detta ska vara möjligt används programräknarna PC som pekar på nästa instruktion i programminnet, PC1 som mellanlagrar tidigare värden för hopp och PC2 som beräknar nya hoppadresser utifrån offset.

4.2.1 Programminne (PM)

Programminnet (PM) består av ett ROM (PM_comp) med 512 rader av 32-bitars-ord. Detta ger i sin tur en adressbredd på 9 bitar, vilket möjliggör 512 olika instruktioner. Allt innehåll som finns i programminnet initieras från konstanten PM_c som innehåller det kompletta Othello-programmet. Minnet tillåter endast läsning och instruktionerna hämtas via indexering med programräknaren.

4.2.2 Registerminne (RM)

Registerminnet implementeras som ett synkront registerfönster med 16 register à 16 bitar (RM_comp). Det adresseras via signalerna IR1 och IR2 och tar emot data antingen från ALU:n eller dataminnet (DM), alternativt från PS/2-tangentbordet. Skrivning sker synkront när write_enable är hög och adressen i IR2 är tillämpad. Skrivning sker inte till register R0 och R15 som har andra funktioner. R0 är konstant noll, medan R15 reserveras för tangentbordsinmatning. När keyboard_interrupt (I-flaggan) är hög skrivs scancode direkt till R15. Det finns också utgångar (regA_out och regB_out) som levererar data från de adresserade registerna.

4.2.5 Aritmetisk-logisk enhet (ALU)

Aritmetiska-logiska enheten är definierad i ALU_comp och tar emot två registervärden (regA, regB), en 16 bitars immediate (digitC) och ett 5 bitars styrfält (control_signals). Operationer som ADD, SUB, MUL, LSL, LSR, AND, OR och NOT stöds av ALU:n och beroende på control_signals(4) multiplexas antingen regB eller digitC in i datavägen. Resultatet D2 samt statusflaggorna Z, N, V, C och I genereras sedan och registreras i en klockad process, vilket möjliggör pipelining. Flaggan C motsvarar bit 16 vid addition och subtraktion i två komplement, V beräknas med XOR-logik, N representerar den mest

signifikanta biten i resultatet, Z sätts om resultatet är noll och I är endast satt om vi ska ta emot en input från tangentbordet.

4.2.6 Kontrollenheten (CU)

Kontrollenheten är implementerad som CU_comp och tar emot instruktioner IR0 till IR2 och genererar styrsignaler som ALU_control_signals, RM_write_enable, DM_write_enable och styr även muxarna, bland annat DM_mux_addr, DM_ALU_result_signal. Genom en with-select-sats framställs rätt signaler baserat på opcode-fältet i instruktionen. CU ansvarar också för adresslägeshantering, alltså om adresser ska hämtas från register eller vara immediate. CU styr även vilken källa som ska skriva data till registerminnet.

4.3 Dataminnet (DM)

Dataminnet använder sig av RAM/ROM (DM_comp) med 512 ord och 16 bitar. Det har också ett tvågränssnitt, ett för CPU (som tillåter både läsning och skrivning) och ett för VGA (endast läsning). CPU:n adresserar minnet via en 2:1-multiplexer som styrs av mux_addr_signal. Det är detta som avgör om adressen kommer från immediatefältet eller ett register. Skrivning sker synkront på klockans stigande flank om styrsignalen cpu_we är hög, då skrivs ett värde Z2 till aktuell adress. Läsningar sker asynkront för både CPU och VGA.

4.4 PS/2-tangentbordet (KBD_comp)

PS/2-tangentbordet synkroniserar den inkommande PS/2-klockan och datan till systemets klocka. Det finns ett shiftregister som används för att ta emot 11 bitar (startbit, 8 databit, paritetsbit, stoppbit). En tillståndsmaskin med tillstånden IDLE, MAKE och BREAK som används för att hantera break-koder (exempelvis 0xF0). När en giltig make-kod detekterats genereras en puls (MAKE_op) och den mottagna scan-koden skickas vidare på utgången key. Därefter skrivs den in i register R15 under nästa klockcykel tillsammans med en puls på keyboard_interrupt.

4.5 VGA-motor

VGA-motorn och tecken-ROM utgörs av vga_comp, som tar emot cellkoder (VM_data) och genererar tillhörande adress (VM_addr), samt VGA signaler som Hsync och Vsync och RGB-utgångar (vga_RGB).

4.5.1 Allmänt om VGA-motorn

Genom frekvensdelning från systemets 100 MHz-klocka erhålls en pixelklocka på 25 MHz. Det finns horisontella och vertikala räknare (från 0-799 och 0-524) som genererar positioneringssignaler. Adressen till videominnet (video RAM) beräknas som rad x 80 + kolumn. Teckenmönstret hämtas från en ROM med 2048 ord och 12 bitar (char_rom) där varje tecken representeras som ett 8x8 pixelmönster. RGB styrs via multiplexing och sätts till noll under blanknings-perioder.

4.5.2 Signalkopplingar och dataströmmar

Signalkopplingar och dataströmmar i systemet hanteras enligt instruktioner som hämtas från PM (programminnet) via PC (programräknaren) och laddas in i registrena IR1/IR2. Systemets styrsignaler

genereras av CU. Det är dessa signaler som i sin tur styr ALU:n, registerminnet och dataminnet. All data från registerminnet matas in i ALU:n och resultatet går sedan via en multiplexer och sedan tillbaka till registerfilen. CU styr även skrivning till dataminnet och även valet av adresskälla. Det finns även en extern input från tangentbordet som skrivs till register R15 och kan läsas av instruktioner. Samtidigt läser VGA-motorn kontinuerligt från dataminnet för att uppdatera skärmen.

4.6 Sammanfattning

Med denna hårdvaruarkitektur uppnås en komplett tvåstegs-pipelinead processor som integrerar med instruktioner, register, aritmetik, minnesoperationer, tangentbordsinmatning och grafikutmatning som är implementerad i VHDL för det ska kunna användas på ett Basys3-kort.

5. Mjukvaran

5.1 Beskrivning

Mjukvaran till Othello-systemet är skriven i ett egenutformat assembler-liknande språk som är anpassat för den egenutvecklade pipeline-CPU:n. Mjukvaran innehåller initialisering av spelbrädet och interaktiv spelstyrning via tangentbord. Kompilering till maskinkod sker med ett egenutvecklat Python-baserat komplieringsverktyg, som översätter . asm-filer till en maskinkodsfil (. txt) som kan laddas in i programminnet (PM).

5.2 Översikt

Vid start initieras brädet till ett tomt läge med åtta rader och kolumner, med fyra startbrickor utplacerade i mitten. Därefter väntar mjukvaran på tangentbordsinmatning, där användaren kan styra en markör med tangenter W, A, S och D och sedan placera ut en bricka med enter.

Eftersom CPU:n inte har ett operativsystem används en avbrottsstyrd modell, där tangenttryckningar från PS/2-tangentbordet genererar keyboard_interrupt, vilket avläses i register R15.

5.3 Instruktionsuppsättning

Instruktionsuppsättningen består av 5-bitars opcodes med stöd för bland annat aritmetik, logik, jämförelser, hopp och avbrottshantering. Instruktionerna består bland annat av LDI, ADD, CMP, BEQ, STO osv. Varje instruktion översätts till ett 32-bitars instruktionsord i maskinkod samtidigt som kompilatorn tolkar etiketter, parsar operander och sätter binära kontrollfält. (Se appendix …)

5.4 Spelbrädesinitialisering

När man startar programmet nollställs alla register som behövs och brädet skrivs in i videominnet (video_ram) som en 8x8-ruta centrerad på en 15x20 matris. Startläge för adressberäkning sker enligt följande formel:

```python
# initialiserar indexen x och y så vi kan starta spelet
def board_index(x, y):
    return (START_Y + y) * VR_WIDTH + (START_X + x)
```

Figur 9.

Detta görs med hjälp av multiplikation och addition mellan register som håller X och Y koordinater. Resultatet används för att adressera video-RAM, där ett värde som representerar en tom ruta (EMPTY_BOX = 1) lagras.

5.5 Interaktion och tangentbordsinmatning

När användaren trycker på en tangent kommer en scankod att läsas in till ett register R15 av tangentbordsmodulen (KBD_comp). Mjukvaran detekterar I-flaggan och hoppar till avbrottshanteraren. Scankoden jämförs sedan med kända värden och uppdaterar markörens position i register R1 (värdet av markörens index). Nedan följer ett exempel i assembler för hur det kan se ut när en tangenttryckning sker:

```
; ------------------------------------------------------------------------
; Program-loop: Kolla om det har skett en tangenttryckning.
; ------------------------------------------------------------------------
main_loop:
    nop            ; om det blir en keyboard_interrupt
    brie interrupt ; gå till interrupt-submodul-grej
    bra main_loop  ; annars loopa vidare
```

Figur 10: Main-loop för alla tangenttryckningar. Denna körs hela tiden och kommer hoppa till interrupt vid en tangenttryckning.

```
; ------------------------------------------------------------------------
; Interrupt-kontroll: Kolla vilken interrupt som har skett.
; ------------------------------------------------------------------------
interrupt:
    cmpi r15, 29       ; om tangenttryckning är 'w'
    nop
    beq move_cursor_up
    cmpi r15, 27       ; om tangenttryckning är 's'
    nop
    beq move_cursor_down
    cmpi r15, 35       ; om tangenttryckning är 'd'
    nop
    beq move_cursor_right
    cmpi r15, 28       ; om tangenttryckning är 'a'
    nop
    beq move_cursor_left
    cmpi r15, 90   ; om tangenttryckning är "enter"
    nop
    beq place_tile
    cmpi r15, 22       ; om tangenttryckning är '1'
    nop
    beq place_white
    cmpi r15, 30       ; om tangenttryckning är '2'
    nop
    beq place_black
    CLI                ; okänd tangenttryckning
    nop
    bra main_loop
```

Figur 11: Vid tangenttryckning triggas I-flaggan och interrupt subrutinen kör.

5.6 Brickplacering och spelmekanik

När användaren trycker på Enter körs en subrutin (place_tile) som placerar en bricka (värde 2 för svart och värde 3 för vit) i aktuell ruta på spelplanen. Detta sker endast om det är ett giltigt drag enligt Othellos regler. Brickans placering sker genom följande kodrader i assembler för Othello programmet:

```
; ------------------------------------------------------------------------
; PLACE TILE
; ------------------------------------------------------------------------
place_tile:
    cmpi r6, 2
    nop
    beq place_white
    cmpi r6, 3
    nop
    beq place_black
    nop
    CLI
    bra move_cursor_home
```

Figur 12: Place Tile i assembler

```
; ----------------------------
; PLACE WHITE TILE
; ----------------------------
place_white:
    nop
    sto r2, r1
    nop
    ldi r13, 0
    nop
```

Figur 13: Place white tile i assembler

```
; ------------------------
; PLACE BLACK TILE
; ------------------------
place_black:
    nop
    sto r2, r1
    nop
    ldi r13, 0
    nop
```

Figur 14: Place black tile i assembler

5.7 Sammanfattning

Mjukvaran i Othello-projektet är ett samspel mellan egenutvecklad assembler, avbrottshantering och tangentbordsstyrt spelgränssnitt. Den möjliggör ett fullt spelbart brädspel där all logik är exekverad direkt på FPGA-baserad CPU. Mjukvaran är dessutom lätt att vidareutveckla genom att lägga till nya instruktioner, förbättra avbrottsrutiner och utöka spelets funktionalitet.

6. Slutsatser

Arbetet med projektet har huvudsakligen gått bra. Något som gick extra bra var testning och simulering. På grund av att varje komponent fick en egen testbänk, kunde de testas enskilt och simuleras i ModelSim. Det tas således med som en lärdom att utförliga komponenttester sparar tid och förenklar det vidare arbetet med att integrera komponenterna i hårdvarusystemet. Även att dela upp projektet i mindre delar, både gällande deadlines och i vilken ordning de olika komponenterna skapas och integreras, tas med som en lärdom för framtiden.

Projektet har flera delar som hade kunnat utvecklas för att uppnå ett bättre resultat. Till exempel fanns planer på att implementera något slags “win-condition” för att datorn själv ska identifiera att en spelare har vunnit och därefter avsluta spelet. Detta sparades dock till eventuell fortsatt utveckling av spelet, då det inte ansågs vara tillräckligt relevant att lägga tid på i projektets slutskede. Något annat som skulle kunna förbättras är att minska antalet “nop-instruktioner” som behövs i programkoden för att spelet ska fungera. Som följd av den tvåstegs-pipeline som processorn utgör, finns inget utrymme för implementering av data-forwarding. Det har därför uppstått så kallade “långa, kritiska vägar”, där vissa instruktioner inte hinner exekveras klart, innan de används av instruktionen efter. Istället för att manuellt behöva lägga in “nop-ar” i den assemblerkod som skrivits för spelet, hade en lösning som exempelvis innebär identifiering av instruktioner med långa kritiska sökvägar i CPU:n, varit önskvärt för ett förbättrat projekt.

Sammanfattningsvis har projektet givit gruppen lärdomar om att skapa en enklare dator från grunden. Projektet har inneburit möjligheten att applicera kunskap från tidigare kurser för att producera något på egen hand.

7. Referenser

7.1 Appendix

7.1.1 VHDL-kod

Projektets källkod finnes i LIU:s GitLab under:
Computer Engineering Course Projects/Computer Hardware and Architecture/2025/G36/kod

Koden innefattar hela datorns konstruktion, som beskrivet i dokumentationen ovan.

7.2.1 Instruktionsuppsättning

Data-förflyttning:

Opkod	**Förkortning**	**Operander**	**Förklaring**	**Flaggor**	**Exempel**
x00	NOP	inga	Gör inget	Inga	“nop”
x01	LD	rR, rA	Ladda rB till rR	Inga	“ld rR, K”
x11	LDI	rR, K	Ladda K till rR	Inga	“ldi rR, K”
x02	STO	rA, rB	Lagra värdet i rA på plats i DM som anges i rB	Inga	“sto rA, rB”
x12	STOI	rA, k	Lagra värdet i rA på plats k i DM	Inga	“stoi rA, k”

Aritmetik:

Opkod	**Förkortning**	**Operander**	**Förklaring**	**Flaggor**	**Exempel**
x03	ADD	rR, rA, rB	Spara i rR summan av rA & rB	Z, N, C, V	“add rR, rA, rB”
x13	ADDI	rR, rA, K	Spara i rR summan av rA & K	Z, N, C, V	“addi rR, rA, K”

x04	SUB	rR, rA, rB	Spara i rR differensen av rA & rB	Z, N, C, V	“sub rR, rA, rB”
x14	SUBI	rR, rA, K	Spara i rR differensen av rA & K	Z, N, C, V	“subi rR, rA, K”
x05	MUL	rR, rA, rB	Spara i rR produkten av rA & rB	Z, N	“mul rR, rA, rB”
x15	MULI	rR, rA, K	Spara i rR produkten av rA & K	Z, N	“muli rR, rA, K”

Bit-operationer:

OpKod	**Förkortning**	**Operander**	**Förklaring**	**Flaggor**	**Exempel**
x06	LSR	rR, rA, rB	Spara i rR bitresultatet av att skifta rA,rB antal gånger	Z,N	"lsr rR, rA"
x16	LSRi	rR, K	Spara i rR bitresultatet av att skifta rA, K antal gånger	Z,N	"lsri rR, rA, K"
x07	AND	rR, rA, rB	Spara i rR bitresultatet av rA 'and' rB	Z,N	"and rR, rA, rB"
x17	ANDi	rR, rA, K	Spara i rR bitresultatet av rA 'and' K	Z,N	"andi rR, rA, K"
x08	OR	rR, rA, rB	Spara i rR bitresultatet av rA 'or' rB	Z,N	"or rR, rA, rB"
x18	ORi	rR, rA, K	Spara i rR bitresultatet av rA 'or' K	Z,N	"ori rR, rA, K"
x09	NOT	rR, rA	Spara i rR bitresultatet av att inverta rA	Z,N	"not rR, rA"

x19	NOTi	rR, K	Spara i rR inverteringen av bitarna talet K representerar	Z,N	"noti rR, K"

Kontrollflöde:

OpKod	**Förkortning**	**Operander**	**Förklaring**	**Flaggor**	**Exempel**
x0A	CMP	rA, rB	Samma som sub fast spara inte resultatet	Inga	"cmp rA, rB"
x1A	CMPi	rA, K	Samma som subi fast spara inte resultatet	Inga	"cmpi rA, K"
x0B	BEQ / JEQ	K	Hoppa till rad K i PM om resultatet var noll	Inga	"beq K"
x0C	BNE / JNE	K	Hoppa till rad K i PM om resultatet INTE var noll	Inga	"bne K"
x0D	BRA / JMP	K	Hoppa till rad K i PM	Inga	"bra K"
x0E	BRIE	K	Hoppa till rad K i PM om I-flaggan är ettställd	Inga	"brie K"
x0F	CLI	-	Nollställ I-flaggan	I	"cli"

7.2.2 Blockscheman

